Infraestrutura de software
Relatório - syncMETROREC

Pedro Vitor Chagas Bedor

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado pede a implementação, em C com threads POSIX (pthread), da sincronização de um sistema de metrô: cada vagão é uma thread e cada passageiro também. É preciso definir a struct station e as funções station_init, station_load_car, station_wait_for_car e station_on_board, usando apenas um mutex e variáveis de condição — sem semáforos e sem busy-wait. O projeto é formado pel código implementado (src/sync_car.h e src/sync_car.c), o runner de teste disponibilizado no Classroom, o Makefile, esse relatório e o evidencias.log.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Sem semáforos nem outras primitivas	OK	sync_car.c usa só as 7 funções pthread listadas
No máximo um lock por struct station	OK	struct station tem um único pthread_mutex_t
Embarque simultâneo (vários retornam antes do on_board)	OK	pthread_cond_broadcast; assento reservado na liberação
Sem busy-wait	OK	toda espera usa pthread_cond_wait
load_car retorna imediato (sem passageiros / count=0)	OK	testes 1 e 2 do runner
Exatamente min(esperando, count) embarcam por viagem	OK	200 execuções sem "Many passengers"
Passa no runner completo	OK	./sync_car → Looks good!

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

Teste de estresse (200 execuções):

```
for i in $(seq 1 200); do ./sync_car > /dev/null 2>&1 || echo
"FALHOU na execucao $i"; done
```

Teste ThreadSanitizer:

```
make tsan
```

```
./sync_car_tsan
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **src/sync_car.h → struct station (mutex, 2 variáveis de condição, 3 contadores) e protótipos das 4 funções**
2. **src/sync_car.c → implementação de station_init, station_load_car, station_wait_for_car e station_on_board**
3. **src/sync_car-runner.c → programa disponibilizado no classroom**
4. **Makefile → compilação (alvos all, tsan e clean)**

Decisões técnicas:

1. **Um único mutex + 2 variáveis de condição (carroChegou / carroPronto) → cumpre a restrição de 1 lock → padrão monitor, sem semáforos**
2. **broadcast pros passageiros e signal pro carro → vários passageiros embarcam de uma vez (requisito 5); só há um carro por vez → sem serialização**
3. **Zerar assentosLivres ao fim do load_car → o carro não aceita passageiro "fantasma" depois de partir → sem deadlock**
4. **struct station no .h (e não no .c) → o runner declara struct station station; na pilha e precisa do tipo completo → compila normalmente com o runner do classroom**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Monitor	Implementação inicial baseada no padrão do problema (mutex + 2 condições, assento reservado na liberação)	Sem troca

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Evidência
1	S1	Estruturei o projeto (pasta src/ + runner copiado)	OK		evidencias.log
2	S1	Implementei sync_car.h (struct + protótipos)	OK		evidencias.log
3	S1	Implementei sync_car.c (as 4 funções)	OK		evidencias.log
4	S1	Compilei com make	Parcial	warning foo vem do runner, não do meu código	evidencias.log
5	S1	Rodei ./sync_car	OK		evidencias.log

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Evidência
6	S1	Estressei com 200 execuções	OK		evidencias.log
7	S1	Rodei ThreadSanitizer	OK	avisos só nas globais do runner (threads_completed, load_car_returned)	evidencias.log
8	S1	Adicionei alvo tsan no Makefile	OK		evidencias.log

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

6.1 Log automático

Log adicionado no Evidencias.log

6.2 Prints

```
syncMETROREC main ? ❱ make
gcc -Wall -Wextra -O2 -pthread -c -o src/sync_car.o src/sync_car.c
gcc -Wall -Wextra -O2 -pthread -c -o src/sync_car-runner.o src/sync_car-runner.c
src/sync_car-runner.c: In function ‘alarm_handler’:
src/sync_car-runner.c:54:24: warning: unused parameter ‘foo’ [-Wunused-parameter]
   54 | void alarm_handler(int foo)
      |                    ~~~~^~~
gcc -Wall -Wextra -O2 -pthread -o sync_car src/sync_car.o src/sync_car-runner.o
```

Figura 1. O comando make vem com um warning referente ao código presente no arquivo disponibilizado pelo classroom (sync_car-runner.c).

```
syncMETROREC main ? ❱ ./sync_car
car entering station with 33 free seats
car departed station with 33 new passenger(s) (expected 33)
car entering station with 37 free seats
car departed station with 37 new passenger(s) (expected 37)
car entering station with 14 free seats
car departed station with 14 new passenger(s) (expected 14)
car entering station with 37 free seats
car departed station with 16 new passenger(s) (expected 16)
Looks good!
```

Figura 2. ./sync_car rodando como esperado



Figura 3. O ThreadSanitizer também resulta em warnings, todos também referentes ao arquivo disponibilizado pelo classroom que não foi modificado (sync_car-runner.c).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Nesta atividade, utilizei o modelo DeepSeek V4 Pro como tutor para me auxiliar inicialmente na parte de entendimento e planejamento do problema. Além de me indicar formas de como testar e colocar a minha implementação em “estresse”.

Prompts principais (liste, sem história):

- **pq ta acontecendo esse erro no meu make??**
- **além do runner, como posso fazer pra testar meu código?**
- **como posso adicionar esse tsan já no meu Makefile?**

O que validei manualmente e como:

Como dito anteriormente, a IA só foi utilizada como uma ajuda na parte de estruturação da implementação, a parte da escrita do código foi feita de forma manual.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:
O projeto saiu de primeira, porque depois de ler o runner ficou claro que a solução era um monitor com um mutex e duas variáveis de condição. O ponto mais sutil foi reservar o assento na liberação do passageiro e não no on_board, se reservasse no on_board, o broadcast acordava todo mundo e embarcava mais gente do que assento. A maior dificuldade não foi o código, e sim em interpretar os detectores: o TSan aponta data race nas globais do runner e o Helgrind dá alarme falso porque deixa o programa lento demais. Se fosse fazer de novo, faria um teste de mesa antes de codar.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Creio que pra essa implementação eu consegui fazer tudo que era necessário. O código compila limpo (só o warning 'foo' do runner, que já veio do classroom), passa no runner com Looks good!, aguenta 200 execuções seguidas sem falhar e o ThreadSanitizer não acusa nenhum data race no meu código. Caso eu tivesse tempo extra, usaria ele basicamente pra continuar testando: rodar o runner mais algumas centenas de vezes, já que o teste é probabilístico e mais repetições ajudam a pegar qualquer condição de corrida rara que uma rodada só não deixa aparecer. Se algum problema aparecesse nesses testes, eu iria atrás da condição do while do load_car ou da ordem de reserva do assento até resolver.